ARTIFICIAL INTELLIGENCE LAB PROGRAMS(1-5)

Program 1: 8-Puzzle Problem using Breadth First Search

Aim

To solve the 8-puzzle problem by finding the shortest sequence of moves from a given initial state to the goal state using Breadth First Search (BFS).

Algorithm

1. Start with the initial state in a queue (with an empty path). The goal state is 1 2 3 / 4 5 6 / 7 8 0, where 0 is the blank tile.
2. Remove the front state from the queue. If it is the goal state, return the path; if it was already visited, skip it, otherwise mark it as visited.
3. Locate the blank tile and generate new states by moving it up, down, left or right (within the 3x3 grid); add each unvisited new state to the queue with its path.
4. Repeat until the goal state is reached, then print the solution steps from the initial state to the goal.

Program

```
from collections import deque

goal=((1,2,3),(4,5,6),(7,8,0))

def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return i,j

def bfs(start):
    q=deque([(start,[])])
    visited=set()

    while q:
        state,path=q.popleft()

        if state==goal:
            return path+[state]

        if state in visited:
            continue
        visited.add(state)

        x,y=find_zero(state)

        for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            nx=x+dx
            ny=y+dy

            if 0<=nx<3 and 0<=ny<3:
                new=[list(r) for r in state]
```

```
            new[x][y],new[nx][ny]=new[nx][ny],new[x][y]
            new=tuple(tuple(r) for r in new)

            if new not in visited:
                q.append((new,path+[state]))

print("Enter Initial State:")
start=[]

for i in range(3):
    row=tuple(map(int,input().split()))
    start.append(row)

start=tuple(start)

ans=bfs(start)

print("\nSolution Steps:")
for s in ans:
    for r in s:
        print(*r)
    print()
```

Output

```
Enter Initial State:
1 2 3
4 0 6
7 5 8

Solution Steps:
1 2 3
4 0 6
7 5 8

1 2 3
4 5 6
7 0 8

1 2 3
4 5 6
7 8 0
```

Result

The 8-puzzle was solved using BFS. The goal state was reached from the given initial state in 2 moves, which is the shortest solution.

Program 2: N-Queens (8-Queens) Problem using Backtracking

Aim

To place N queens on an N x N chessboard such that no two queens attack each other, using the backtracking technique.

Algorithm

1. Create an N x N board filled with 0 and begin placing queens from the first column.
2. For the current column, try each row and check whether the position is safe (no queen in the same row to the left, or on the upper-left and lower-left diagonals).
3. If the position is safe, place the queen and recursively solve the next column; if all N columns are filled, a solution has been found.
4. If no row in a column is safe, backtrack by removing the previous queen and trying the next row; print the board if a solution exists, otherwise print "No Solution".

Program

```
N = int(input("Enter number of Queens: "))

board = [[0] * N for _ in range(N)]

def safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col >= N:
        return True

    for i in range(N):
        if safe(i, col):
            board[i][col] = 1
            if solve(col + 1):
                return True
            board[i][col] = 0
    return False

if solve(0):
```

```
    print("Solution:")
    for row in board:
        print(row)
else:
    print("No Solution")
```

Output

```
Enter number of Queens: 8
Solution:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

Result

Eight queens were placed on the 8 x 8 board using backtracking such that no two queens attack each other.

Program 3: Water Jug Problem

Aim

To measure a required quantity of water using two jugs of given capacities by repeatedly filling, emptying and pouring between them.

Algorithm

1. Read the capacities of Jug 1 (a) and Jug 2 (b) and the target amount; start with both jugs empty at state (0, 0).
2. If Jug 1 is empty, fill it completely; otherwise, if Jug 2 is full, empty Jug 2.
3. Otherwise, pour water from Jug 1 into Jug 2 until Jug 1 becomes empty or Jug 2 becomes full.
4. Print the state (x, y) after every step and repeat until either jug contains the target amount.

Program

```
a = int(input("Enter capacity of Jug1: "))
b = int(input("Enter capacity of Jug2: "))
target = int(input("Enter target amount: "))

x = 0
y = 0

print("(0,0)")

while x != target and y != target:
    if x == 0:
        x = a
    elif y == b:
        y = 0
    else:
        t = min(x, b - y)
        x -= t
        y += t
    print((x, y))
```

Output

```
Enter capacity of Jug1: 4
Enter capacity of Jug2: 3
Enter target amount: 2
(0,0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
```

Result

Using jugs of capacity 4 and 3, exactly 2 units of water were measured in Jug 1, reaching the state (2, 3).

Program 4: Cryptarithmetic Problem (SEND + MORE = MONEY)

Aim

To solve the cryptarithmetic puzzle SEND + MORE = MONEY by assigning a unique digit to each letter so that the addition is correct.

Algorithm

1. Take the distinct letters S, E, N, D, M, O, R, Y and generate every permutation of 8 different digits from 0 to 9.
2. Reject an assignment if the leading letter S or M is 0.
3. For each valid assignment, compute the values of SEND, MORE and MONEY.
4. If SEND + MORE equals MONEY, print the solution and stop; otherwise try the next permutation.

Program

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for P in permutations(range(10), len(letters)):
   S, E, N, D, M, O, R, Y = P

   if S != 0 and M != 0:  # Leading digits cannot be zero
      send = S * 1000 + E * 100 + N * 10 + D
      more = M * 1000 + O * 100 + R * 10 + E
      money = M * 10000 + O * 1000 + N * 100 + E * 10 + Y

      if send + more == money:
         print("Solution Found")
         print("SEND =", send)
         print("MORE =", more)
         print("MONEY =", money)
         break
```

Output

```
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
```

Result

The puzzle was solved: S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2, giving 9567 + 1085 = 10652.

Program 5: Missionaries and Cannibals Problem

Aim

To transfer 3 missionaries and 3 cannibals from the left bank to the right bank of a river using a boat, without missionaries ever being outnumbered by cannibals on either bank, using BFS.

Algorithm

1. Represent a state as (m, c, b): missionaries and cannibals on the left bank and the boat side (1 = left, 0 = right). The start state is (3, 3, 1) and the goal state is (0, 0, 0).
2. The boat can carry 1 or 2 people: moves are (1M), (2M), (1C), (2C) and (1M, 1C). Apply a move by subtracting from the left bank if the boat is on the left, otherwise adding to it.
3. Accept a new state only if the counts are between 0 and 3 and missionaries are not outnumbered by cannibals on either bank (unless there are no missionaries on that bank).
4. Use a BFS queue with a visited set; the first time the goal is reached gives the shortest path, which is then printed.

Program

```
from collections import deque

start = (3, 3, 1)
goal = (0, 0, 0)

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

def valid(m, c):
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)

def bfs():
    q = deque([(start, [])])
    visited = set()

    while q:
        (m, c, b), path = q.popleft()

        if (m, c, b) == goal:
            return path + [(m, c, b)]

        if (m, c, b) in visited:
            continue
        visited.add((m, c, b))

        for dm, dc in moves:
            if b == 1:
                nm, nc, nb = m - dm, c - dc, 0
            else:
                nm, nc, nb = m + dm, c + dc, 1

            if 0 <= nm <= 3 and 0 <= nc <= 3 and valid(nm, nc):
                q.append(((nm, nc, nb), path + [(m, c, b)]))

ans = bfs()
```

```
print("Solution:")
for s in ans:
    print(s)
```

Output

```
Solution:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

Result

All 3 missionaries and 3 cannibals were safely moved to the other bank in 11 crossings, which is the shortest solution found by BFS.